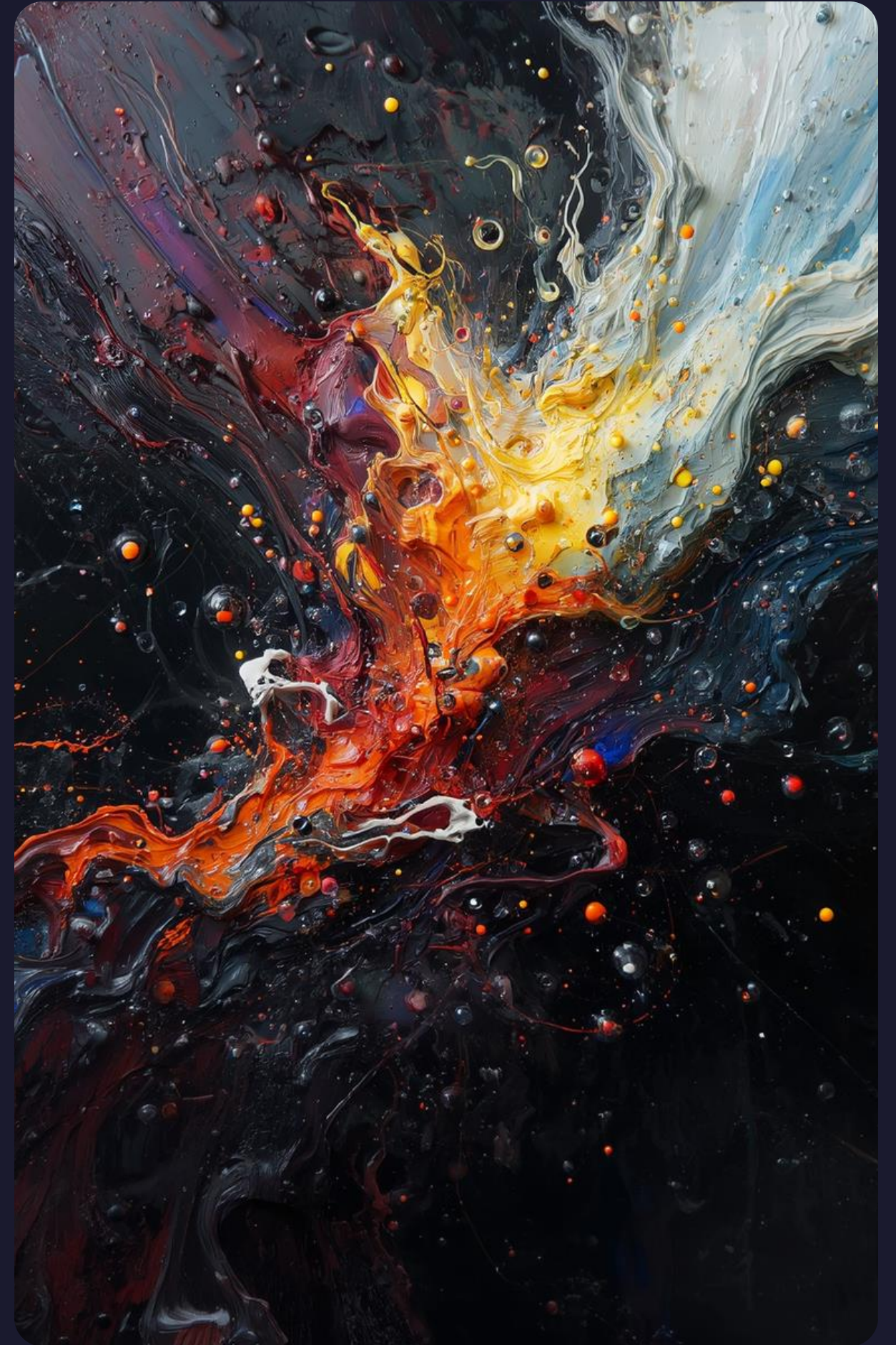
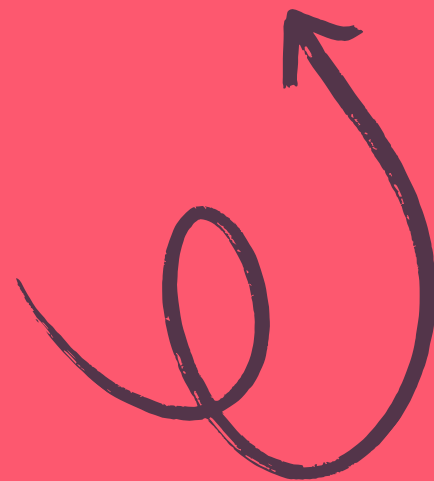


Programe de procesare generică - *„Compute Shaders”*

Cristian Lambru: 2025-26





Cuprins

1. Introducere

2. Sintaxă

3. Structură

4. Tipuri de date

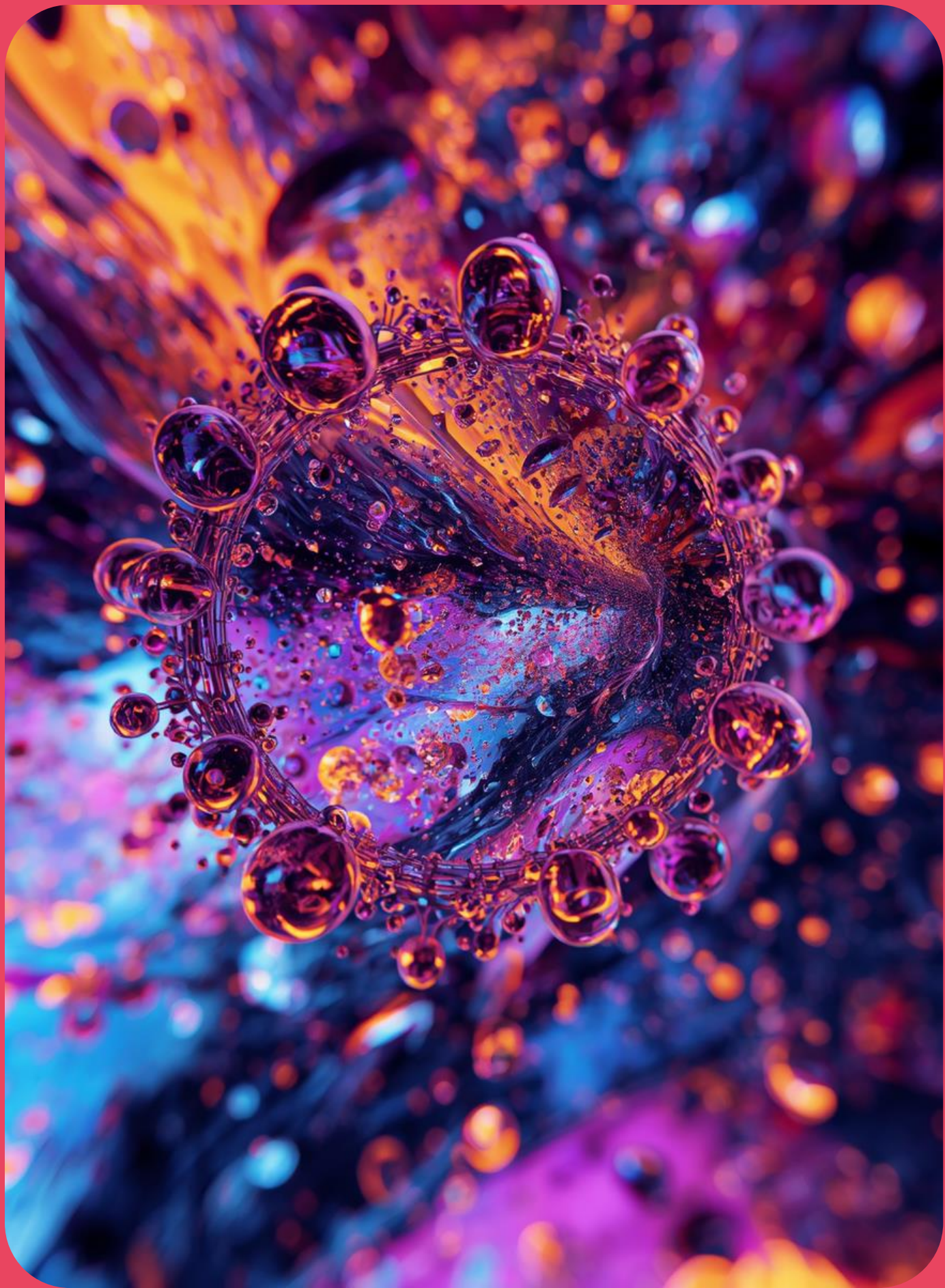
a. Exemplu (I) - Procesul de netezire gaussiană - *Gaussian Blur*

5. Operații atomice

a. Exemplu (II) - Numărarea pixelilor cu o anumită proprietate

6. Memorie partajată

a. Exemplu (III) - Optimizări

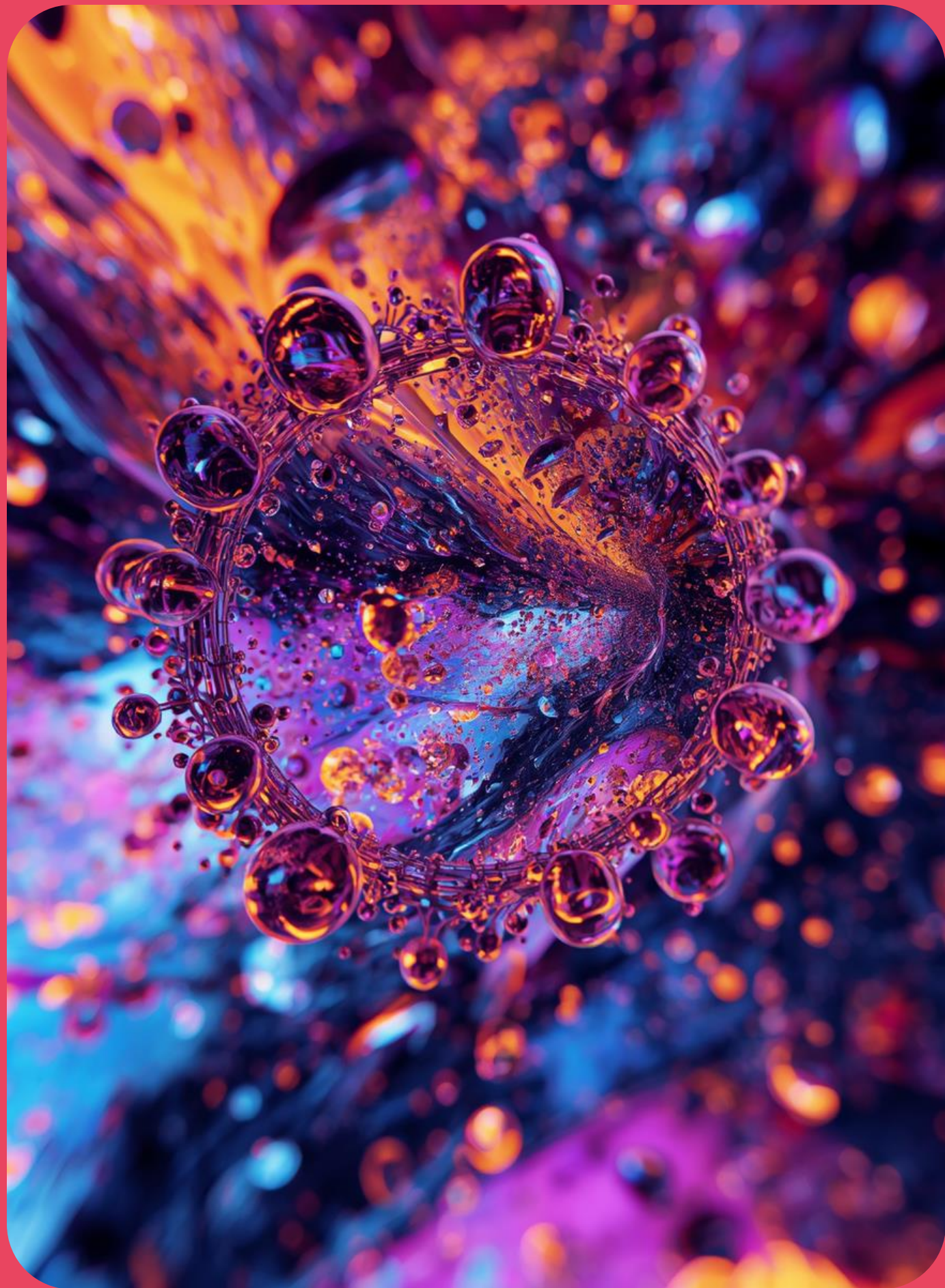


Introducere (I)

Programele de procesare generică, cunoscute în limba engleza sub numele de „*Compute Shaders*”, sunt programe executate pe unitatea de procesare grafică.

Procesorul grafic pornește în paralel mai multe instanțe ale unui program de tip Compute Shader, instanțe executate pe nucleele disponibile de pe dispozitiv.

Nucleele utilizate sunt aceleași care sunt folosite de către unitatea de procesare grafică pentru execuția programelor de tip *Vertex Shader* sau *Fragment Shader*.

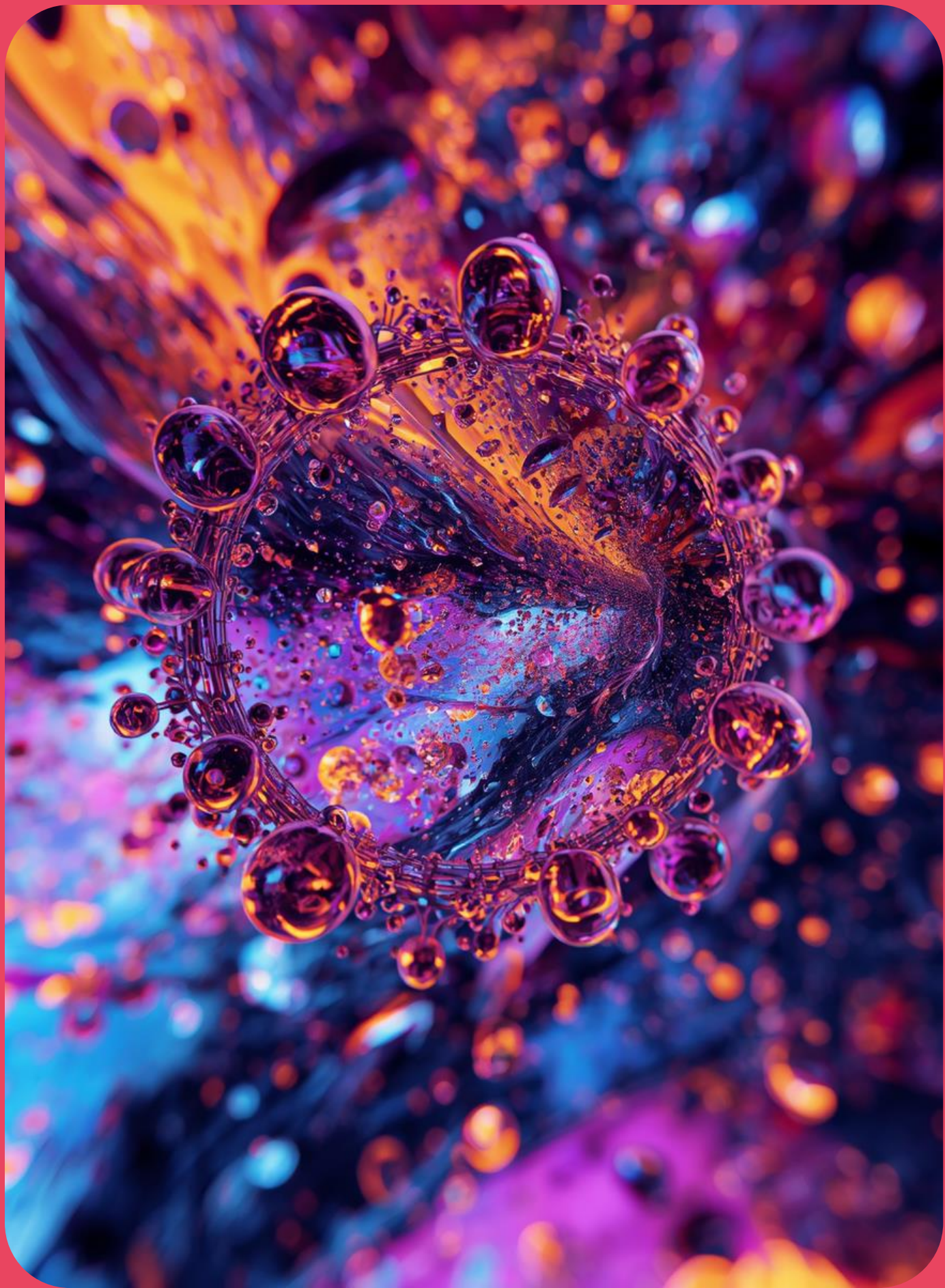


Introducere (II)

Programele de procesare generică au o bandă de procesare specifică în unitățile de procesare grafică, diferită de banda de rasterizare.

Aceste programe primesc la intrare date generice și introduc rezultatul în structuri de date care pot stoca date generice.

Unul dintre avantajele majore oferite de către acest tip de programe este faptul că ele pot fi executate în paralel cu banda de rasterizare.



Introducere (III)

Programele de procesare generică au fost introduse în unitățile de procesare grafică după anul 2006.

Bazat pe API:

- OpenGL a introdus programele de procesare generică în anul 2012;
- Direct3D a introdus în anul 2009.

Sintaxă (I)

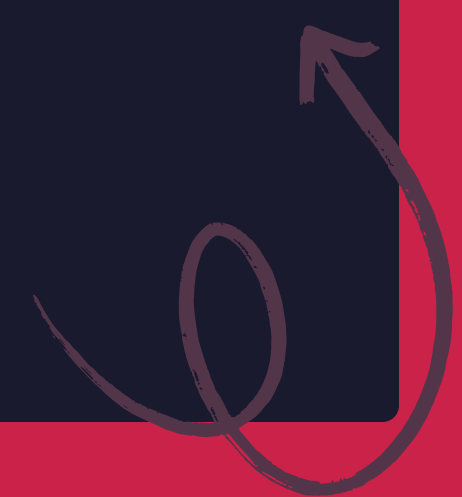
În limbajul **GLSL**, sintaxa este următoarea:

```
#version 430

layout(local_size_x = 8, local_size_y = 8, local_size_z = 8) in;

void main()
{
    uvec3 id = gl_GlobalInvocationID;

    float rand = fract(
        id.x * 1231.3213 +
        id.y * 1231.3212 +
        id.z * 1231.1231
    );
}
```



Sintaxă (II)

```
layout(local_size_x = 8, local_size_y = 8, local_size_z = 8) in;
```

Se lansează un set de 83 fire de execuție. Nu toate firele de execuție este obligatoriu să fie în paralel la același moment. Unitățile de procesare grafică au anumite **limitări**.

Pornirea unui Compute Shader este realizată de către **unitatea centrală de procesare**. În API-ul grafic **OpenGL**, pornirea se realizează cu următoarea directivă:

```
void glDispatchCompute(GLuint num_groups_x, GLuint num_groups_y, GLuint num_groups_z);
```



Sintaxă (III)

Astfel, utilizarea următoarei directive:

```
glDispatchCompute(2, 2, 2);
```

Lansează $2^3 * 8^3 = 16^3$ fire de execuție.



Structură

Sistemul programelor de tip Compute Shader pun la dispoziție o structură de paralelizare formata din *grupuri de lucru*, cunoscute în limba engleză sub numele de „*Work Groups*” și *fire de execuție locală* la *nivelul unui grup de lucru*.

Suplimentar, structura grupurilor de lucru este în **3 dimensiuni**. Această repartiție în 3 dimensiuni are scopul să se plieze corespunzător pe structurile de date utilizate de către unitatea de procesare grafică.

Astfel, următoarea structurare creează 16 fire de execuție pe axa X, 16 fire pe axa Y și 16 fire pe axa Z.

```
glDispatchCompute(2, 2, 2);
```

```
layout(local_size_x = 8, local_size_y = 8, local_size_z = 8) in;
```



Sintaxa (IV)

În limbajul **GLSL**, există următoarele attribute implicite.

```
in uvec3 gl_NumWorkGroups;  
in uvec3 gl_WorkGroupID;  
in uvec3 gl_LocalInvocationID;  
in uvec3 gl_GlobalInvocationID;  
in uint  gl_LocalInvocationIndex;
```



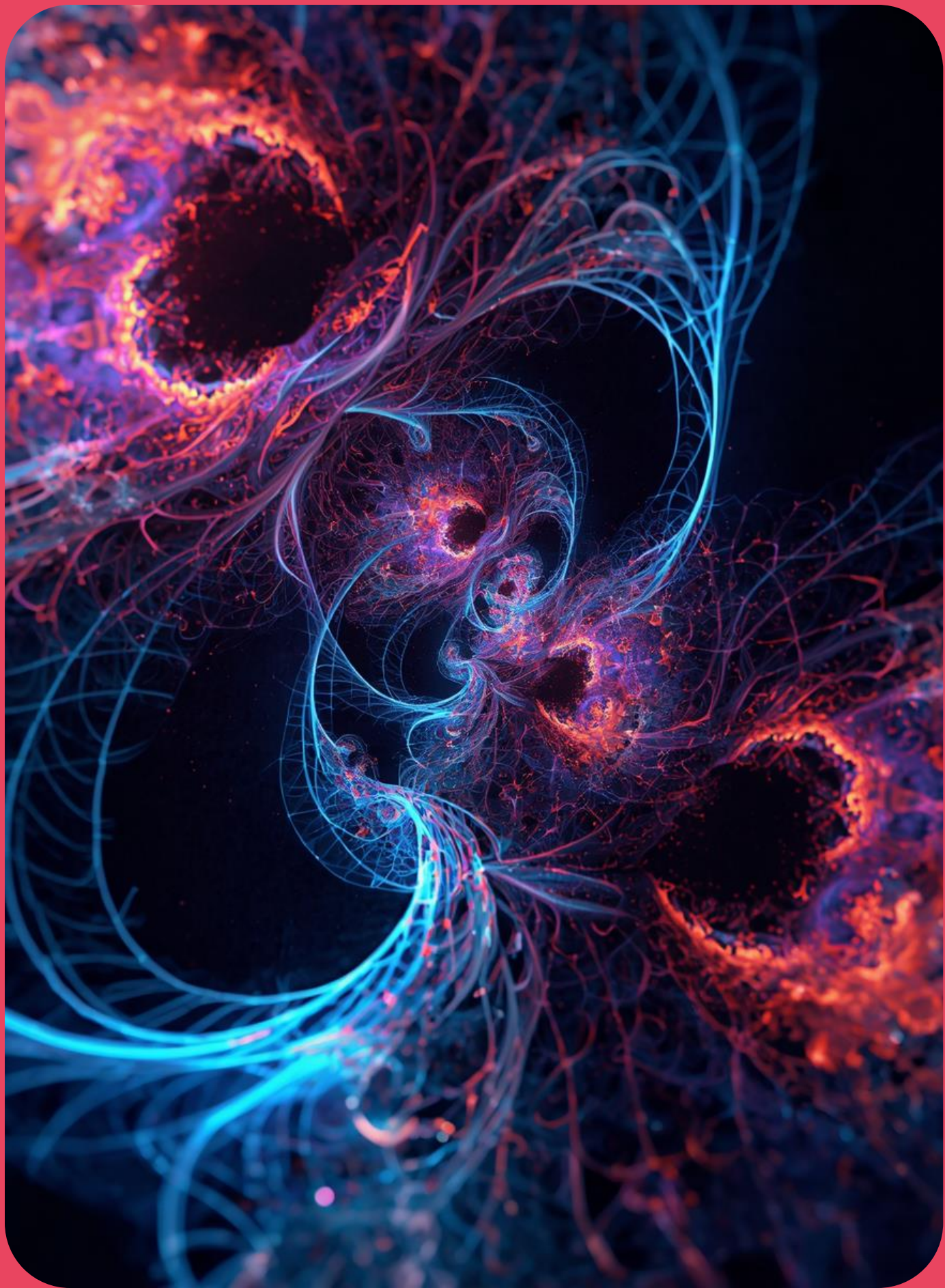
Sintaxă (V)

Formele de calcul pentru valorile atributelor implicite menționate anterior sunt următoarele:

```
gl_GlobalInvocationID = gl_WorkGroupID * gl_WorkGroupSize + gl_LocalInvocationID;
```

```
gl_LocalInvocationIndex =  
    gl_LocalInvocationID.z * gl_WorkGroupSize.x * gl_WorkGroupSize.y +  
    gl_LocalInvocationID.y * gl_WorkGroupSize.x +  
    gl_LocalInvocationID.x;
```





Tipuri de date

Există 2 tipuri de date de intrare/ieșire din/în care se pot prelua/introduce date.

- *Imagini*;
- Structuri de tip „*Shader Storage Buffer Object*”.

Tipuri de date - Imagini

Imaginile sunt similare cu obiectele de tip textură.

```
#version 430

layout(local_size_x = 8, local_size_y = 8, local_size_z = 1) in;
layout(rgba32f, binding = 0) uniform image2D destImage;

void main()
{
    ivec2 id = ivec2(gl_GlobalInvocationID);

    float rand = fract(
        float(id.x) * 1231.3213 +
        float(id.y) * 1231.3212
    );

    imageStore(destImage, id, vec4(rand));
}
```

Image load store - opengl wiki. Available at: https://wikis.khronos.org/opengl/Image_Load_Store (Accessed: 22 April 2026).



Exemplu (I) - Procesul de netezire gaussiană (I)

```
#version 430

layout(local_size_x = 16, local_size_y = 16, local_size_z = 1) in;
layout(rgba32f, binding = 0) uniform readonly image2D srcImage;
layout(rgba32f, binding = 1) uniform writeonly image2D dstImage;

const float kernel[5][5] = float[5][5](
    float[5](1.0, 4.0, 6.0, 4.0, 1.0),
    float[5](4.0, 16.0, 24.0, 16.0, 4.0),
    float[5](6.0, 24.0, 36.0, 24.0, 6.0),
    float[5](4.0, 16.0, 24.0, 16.0, 4.0),
    float[5](1.0, 4.0, 6.0, 4.0, 1.0)
);

void main()
{
    ivec2 size = imageSize(srcImage);
    ivec2 p = ivec2(gl_GlobalInvocationID.xy);
```

$$\frac{1}{256} \cdot \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

Exemplu (I) - Procesul de netezire gaussiană (II)

```
void main()
{
    ivec2 size = imageSize(srcImage);
    ivec2 p = ivec2(gl_GlobalInvocationID.xy);

    if (p.x >= size.x || p.y >= size.y) {
        return;
    }

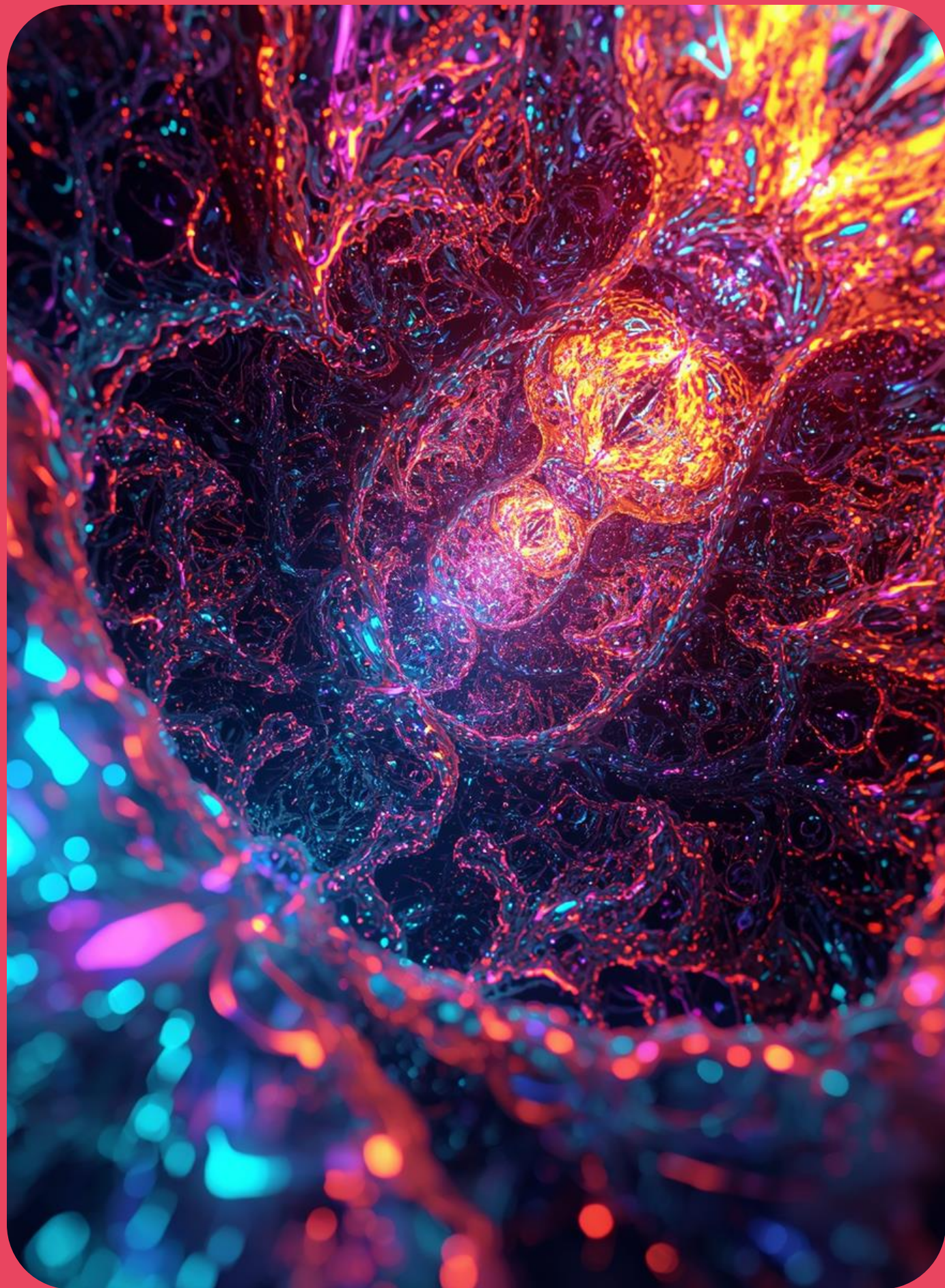
    vec4 sum = vec4(0.0);

    for (int ky = -2; ky <= 2; ++ky) {
        for (int kx = -2; kx <= 2; ++kx) {
            ivec2 samplePos = clamp(p + ivec2(kx, ky), ivec2(0), size - 1);
            float w = kernel[ky + 2][kx + 2];
            sum += imageLoad(srcImage, samplePos) * w;
        }
    }

    imageStore(dstImage, p, sum / 256.0);
}
```

$$\frac{1}{256} \cdot \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$





Operații atomice (I)

Deoarece **mai multe instanțe** ale unui program de tip Compute Shader pot fi **executate la același moment pe mai multe fire de execuție**, poate apărea situația în care **2 sau mai multe instanțe** încearcă să **introducă informație la aceeași zonă de memorie**.

Din acest motiv, unitatea de procesare grafică **permite realizarea** unui set limitat de **operații atomice** pe imagini și pe structurile de tip SSBO.

Operații atomice (II)

```
int atomicAdd(inout int mem, int data);  
int atomicMin(inout int mem, int data);  
int atomicMax(inout int mem, int data);  
int atomicAnd(inout int mem, int data);  
int atomicOr(inout int mem, int data);  
int atomicXor(inout int mem, int data);  
int atomicExchange(inout int mem, int data);  
int atomicCompSwap(inout int mem, int compare, int data);
```

Exemplu (II) - Numararea pixelilor de culoare roșie

```
#version 430

layout(local_size_x = 16, local_size_y = 16, local_size_z = 1) in;

layout(rgba32f, binding = 0) uniform readonly image2D srcImage;

layout(std430, binding = 1) buffer CounterBuffer
{
    uint redPixelCount;
};

void main()
{
    ivec2 size = imageSize(srcImage);
    ivec2 p = ivec2(gl_GlobalInvocationID.xy);

    if (p.x >= size.x || p.y >= size.y) {
        return;
    }

    vec4 color = imageLoad(srcImage, p);

    if (color.r > 0.5 && color.g < 0.5 && color.b < 0.5) {
        atomicAdd(redPixelCount, 1u);
    }
}
```





Memorie partajată

Există posibilitatea sa se creeze **attribute** care sunt **partajate de toate firele de execuție** pornite pentru instanțele din **același grup de lucru**.

Suplimentar, uneori este necesar să se realizeze pași secvențiali la nivelul firelor de execuție din același grup de lucru, astfel că există următoarea directivă, care blochează toate firele de execuție astfel încât să așteapte realizarea pașilor anteriori directivei.

```
barrier();
```

Exemplu (III) - Optimizări (I)

```
#version 430

layout(local_size_x = 16, local_size_y = 16, local_size_z = 1) in;

layout(rgba32f, binding = 0) uniform readonly image2D srcImage;

layout(std430, binding = 1) buffer CounterBuffer {
    uint redPixelCount;
};

shared uint localCounts[16 * 16];

void main()
{
    ivec2 size = imageSize(srcImage);
    ivec2 p = ivec2(gl_GlobalInvocationID.xy);

    uint lid = gl_LocalInvocationIndex;

    if (p.x < size.x && p.y < size.y) {
        vec4 color = imageLoad(srcImage, p);

        if (color == vec4(1.0, 0.0, 0.0, 1.0)) {
            localCounts[lid] = 1;
        }

        if (color != vec4(1.0, 0.0, 0.0, 1.0)) {
            localCounts[lid] = 0;
        }
    }

    barrier();
}
```



Exemplu (III) - Optimizări (II)

```
barrier();

if ((lid % 16u) == 0u) {
    uint sum = 0u;

    for (uint i = 0u; i < 16u; ++i) {
        sum += localCounts[lid + i];
    }

    localCounts[lid] = sum;
}

barrier();

if (lid == 0u) {
    uint sum = 0u;

    for (uint i = 0u; i < 256u; i += 16u) {
        sum += localCounts[i];
    }

    atomicAdd(redPixelCount, sum);
}
```

